

Gaussian Splatting Revisited: A Functional Taxonomy and Comparative Review of 3DGS Methods

1 Abstract

This section answers: What problem does this survey solve, and what are its main contributions?

This survey claims that a functional taxonomy reveals unifying principles, trade-offs, and open challenges across diverse 3D Gaussian Splatting methods.

A concise 150–250 word summary:

- Context: Neural rendering → need for fast, high-quality reconstruction Key idea: GS replaces discrete surfaces with continuous anisotropic Gaussians Why a survey: Rapid progression (2023–2025), many variants Contributions:
 - Categorization of GS methods
 - Comparison of quality, speed, memory, stability
 - Applications (graphics, robotics, AR/VR, NeRF acceleration)
 - Open challenges & future research directions

2 Introduction

This section answers: Why did 3D Gaussian Splatting emerge, and why is a new survey needed now?

This section claims that recent methodological divergence and application expansion necessitate a principled reorganization of the field.

2.1 Background and Rise of 3DGS

This subsection answers: What limitations of earlier neural rendering pipelines motivated the rise of 3DGS? This subsection claims that inefficiencies and convergence limitations in NeRF-style methods directly motivated explicit Gaussian representations.

Introduce 3D Gaussian Splatting and why it disrupted NeRF-style rendering.

2.2 From Neural Fields to Gaussian Splatting

This subsection answers: What conceptual shift occurs when moving from implicit neural fields to explicit Gaussian primitives?

This subsection claims that explicit, differentiable primitives fundamentally alter the speed–quality trade-off in neural rendering.

Explain the conceptual shift from implicit fields to explicit Gaussian primitives.

2.3 What This Survey Covers and How It Differs

This subsection answers: Which gaps in prior GS surveys does this paper address, and what is the organizing principle of this survey?

This subsection claims that existing surveys lack a functional taxonomy capable of comparing methods across

representation, rendering, and optimization dimensions.

Clearly state how this survey differs from prior GS surveys.

2.4 Paper Organization

This subsection answers: How is the rest of the paper structured to support systematic comparison across 3DGS methods?

This subsection claims that the chosen structure enables both horizontal (within-category) and vertical (cross-category) comparisons.

Summarize what each section covers.

3 A Unified Taxonomy of Gaussian Splatting Research

This section answers: Along which functional dimensions do 3DGS methods differ, and how can we categorize the literature accordingly?

This section claims that most GS methods can be decomposed into a small set of orthogonal functional dimensions.

3.1 Functional Dimensions of 3DGS

This subsection answers: What are the orthogonal design axes that jointly characterize most 3DGS methods? This subsection claims that representation, rendering, optimization, and application extensions form a sufficient taxonomy basis.

3.1.1 Primitive Design

This subsection answers: What Gaussian parameters and primitive choices most directly determine representational capacity and stability?

This subsection claims that primitive parameterization strongly constrains achievable fidelity and optimization robustness.

3.1.2 Rendering / Splatting

This subsection answers: Which rendering and compositing choices govern visibility, aliasing, and efficiency in splatting?

This subsection claims that splatting design is the dominant factor in runtime performance.

3.1.3 Optimization

This subsection answers: Which training objectives, schedules, and regularizers most strongly affect convergence and quality?

This subsection claims that optimization strategy is as critical as representation design for final quality.

3.1.4 Dynamic Extensions

This subsection answers: How do 3DGS methods represent time, motion, and deformation, and what constraints do they impose?

This subsection claims that dynamic modeling introduces new trade-offs between temporal coherence and computational cost.

3.1.5 Editing / Manipulation

This subsection answers: What post-training controls enable editing geometry or appearance in a Gaussian scene representation?

This subsection claims that explicit primitives enable forms of editing not feasible in implicit representations.

3.1.6 Multimodal & Geometry-aware

This subsection answers: How do 3DGS pipelines incorporate geometry priors or multimodal signals?

This subsection claims that multimodal constraints significantly improve robustness and downstream usability.

3.1.7 Compression / Efficiency

This subsection answers: What techniques reduce memory and runtime cost while preserving render quality?

This subsection claims that compression-aware design is essential for real-world deployment.

3.2 Placement of Existing Works Within the Taxonomy

This subsection answers: How do major research directions cluster when mapped into the proposed taxonomy?

This subsection claims that apparent methodological diversity conceals recurring design patterns.

Discuss these as groups, not individual papers:

- Original / Foundational GS
- Quality Improvements
- Dynamic Gaussians / 4D GS
- Editing
- Semantic / Multimodal GS
- Compression
- Large-scale GS
- SLAM / Robotics GS

4 Gaussian Primitive Design

This section answers: How should Gaussian primitives be parameterized and extended to improve fidelity while maintaining stability and efficiency?

This section claims that careful primitive design yields disproportionate gains in reconstruction quality by shaping both representational capacity and optimization behavior.

At the core of 3D Gaussian Splatting lies the choice of primitive used to represent scene geometry and appearance. Unlike implicit neural fields, which encode structure indirectly through network weights, Gaussian splatting relies on explicit, differentiable primitives whose parameterization directly determines what geometric detail can be captured, how appearance varies with view direction, and how efficiently the scene can be optimized and rendered. As a result, the design of the Gaussian primitive plays a central role in balancing visual fidelity, numerical stability, and computational cost.

The original 3DGS formulation adopts a compact yet expressive parameterization based on anisotropic Gaussians with view-dependent appearance, providing a strong baseline that supports real-time rendering and stable optimization. However, this baseline also exposes inherent limitations: first-order local surface approximation, fixed falloff behavior, and sensitivity to covariance degeneracy in complex scenes. These limitations have motivated a growing body of work that extends or constrains the primitive itself, rather than modifying the rendering pipeline or training procedure.

This section organizes the literature on Gaussian primitive design into three complementary directions. Standard parameterization establishes the baseline representation and clarifies the roles of geometric and appearance parameters. Enhanced Gaussian representations increase expressiveness or robustness by modifying the primitive shape, falloff, or covariance structure while preserving the core splatting pipeline. Hybrid representations combine Gaussians with auxiliary primitives, such as meshes, voxel grids, or implicit fields, to introduce additional geometric structure or regularization. Together, these approaches demonstrate that relatively small changes at the primitive level can yield substantial improvements in reconstruction quality, stability, and scalability.

Finally, this section culminates in a comparative summary that highlights the trade-offs among competing primitive designs. By examining these choices through the lens of expressiveness, optimization stability, and efficiency, we show that no single primitive dominates across all scenarios. Instead, effective primitive design is inherently application-dependent, underscoring its importance as a first-order design decision in modern Gaussian splatting systems.

4.1 Standard 3D Gaussian Parameterization

This subsection answers: What is the baseline 3DGS parameterization and which elements are responsible for geometry and appearance?

This subsection claims that the original parameterization balances expressiveness and optimization tractability by separating geometric structure from appearance modeling.

The baseline 3D Gaussian Splatting (3DGS) formulation represents a scene as a collection of explicit anisotropic Gaussian primitives, each encoding local ge-

ometry and appearance in a fully differentiable manner. Each Gaussian is parameterized by a 3D mean that determines its spatial location, a covariance matrix that controls its shape and orientation, an opacity term that governs visibility during compositing, and appearance coefficients that model radiance. This design enables efficient rasterization while maintaining sufficient flexibility to approximate complex surfaces.

Formally, each Gaussian i is defined by parameters

$$\{\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i, \alpha_i, \mathbf{c}_i\},$$

where $\boldsymbol{\mu}_i \in \mathbb{R}^3$ denotes the mean position, $\boldsymbol{\Sigma}_i \in \mathbb{R}^{3 \times 3}$ is a symmetric positive-definite covariance matrix encoding anisotropic spatial extent, and $\alpha_i \in [0, 1]$ is the opacity. The appearance term $\mathbf{c}_i$ consists of spherical harmonic (SH) coefficients that model view-dependent color as a function of viewing direction.

The contribution of a Gaussian at a 3D point $\mathbf{x}$ is given by

$$G_i(\mathbf{x}) = \alpha_i \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1}(\mathbf{x} - \boldsymbol{\mu}_i)\right),$$

with color obtained by evaluating the SH coefficients along the view direction and compositing contributions from multiple Gaussians using alpha blending.

This parameterization cleanly separates geometry (mean and covariance) from appearance (SH coefficients), enabling stable joint optimization using gradient-based methods. While limited to first-order local surface approximation and fixed Gaussian falloff, the baseline design provides a strong balance between expressive power, numerical stability, and computational efficiency, forming the foundation upon which subsequent enhanced and hybrid representations build.

4.2 Enhanced Gaussian Representations

This subsection answers: Which modifications to the primitive improve expressiveness or reduce artifacts? This subsection claims that extending the primitive shape family improves surface approximation and stability beyond what standard Gaussian splats can provide.

The original 3D Gaussian Splatting formulation represents scene geometry using anisotropic Gaussian primitives whose shape and orientation are controlled by a covariance matrix. While this parameterization is efficient and differentiable, its expressive capacity is limited by the assumption of a fixed quadratic falloff and unconstrained covariance optimization. As a result, baseline Gaussians may oversmooth high-curvature regions, blur fine detail in dense areas, or suffer from numerical instability during long training runs.

Enhanced Gaussian representations address these limitations by extending the primitive shape family or by introducing additional constraints on how Gaussians are parameterized and optimized. These extensions operate at the level of individual primitives and therefore preserve the core splatting pipeline, making them attractive

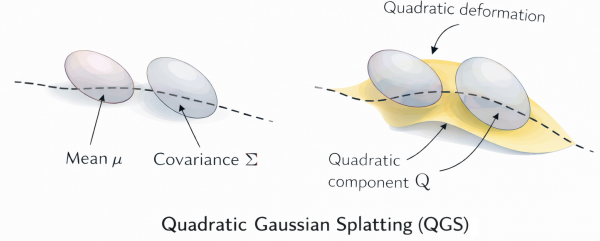


Figure 1: Conceptual comparison between standard Gaussian splatting (first-order local approximation) and Quadratic Gaussian Splatting (QGS), where quadratic terms enable improved curvature modeling with fewer primitives.

drop-in improvements. Broadly, such enhancements aim to either increase local expressiveness—allowing a single primitive to better approximate curved or high-frequency structure—or to improve robustness by preventing degenerate or ill-conditioned splats.

The following subsections examine three representative directions within this design space. Quadratic Gaussian Splatting (QGS) increases the local approximation order of each primitive to better capture curvature. Generalized Exponential Splatting (GES) modifies the falloff function to control smoothing behavior and preserve detail in dense regions. High-rank or stabilized covariance representations constrain the covariance parameterization to improve numerical conditioning and training stability. Together, these approaches illustrate how carefully extending or regulating the Gaussian primitive can yield substantial quality and robustness gains without abandoning the efficiency of explicit splatting.

4.2.1 Quadratic Gaussian Splatting (QGS)

This subsection answers: How does quadratic modeling change the local approximation power? This subsection claims that QGS improves curvature modeling at moderate computational cost.

Quadratic Gaussian Splatting (QGS) extends standard 3D Gaussian Splatting by augmenting each Gaussian primitive with second-order spatial variation, enabling local approximation of curved surface patches rather than purely first-order (locally planar) regions. In the original 3DGS formulation, each primitive contributes radiance through an anisotropic Gaussian defined by a mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$, which can oversmooth high-curvature geometry and force additional densification. Figure 1 illustrates the key idea: quadratic terms increase per-primitive expressiveness so fewer primitives are needed to capture curvature.

The standard Gaussian contribution at a 3D point $\mathbf{x}$ can be written as

$$G(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right).$$

QGS enriches this formulation by introducing an additional quadratic term in the local coordinate frame:

$$G_{\text{QGS}}(\mathbf{x}) = \exp \left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}) - (\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{Q}(\mathbf{x} - \boldsymbol{\mu}) \right).$$

where $\mathbf{Q}$ is a learned symmetric matrix that encodes second-order structure and improves the ability of a single splat to represent curvature. In practice, the quadratic parameters are optimized jointly with $(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, opacity, and appearance, preserving the differentiable rasterization pipeline while adding only a modest number of parameters per primitive. Compared to simply increasing Gaussian count, QGS trades a small increase in per-splat computation for improved geometric fidelity, but benefits from regularization (e.g., conditioning $\mathbf{Q}$ or limiting its magnitude) to avoid instability.

4.2.2 Generalized Exponential Splatting (GES)

This subsection answers: How does changing the falloff function affect quality and optimization?

This subsection claims that generalized exponentials improve detail preservation in dense regions by providing more flexible control over spatial attenuation.

Generalized Exponential Splatting (GES) modifies the standard Gaussian falloff used in 3D Gaussian Splatting to better balance smoothness and detail preservation. In the original formulation, the exponential quadratic decay of the Gaussian can oversmooth densely populated regions, causing fine geometric or appearance details to blur unless compensated by aggressive densification. GES addresses this limitation by generalizing the falloff function, allowing sharper or heavier-tailed attenuation profiles that better preserve high-frequency structure.

In standard 3DGS, the contribution of a Gaussian at a point $\mathbf{x}$ is given by

$$G(\mathbf{x}) = \exp \left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}) \right).$$

GES generalizes this formulation by replacing the quadratic exponent with a parametric falloff:

$$G_{\text{GES}}(\mathbf{x}) = \exp \left(-((\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}))^p \right),$$

where the exponent $p > 0$ controls the sharpness of the attenuation. Values $p > 1$ produce steeper decay that preserves local detail, while $p < 1$ yields heavier tails that improve overlap and robustness in sparse regions.

In practice, GES improves reconstruction fidelity in areas with dense geometry or high-frequency appearance by reducing excessive smoothing, but it also alters gradient magnitudes and optimization dynamics. As a result, GES often requires careful parameter tuning or regularization to maintain stability. Compared to higher-order primitives such as QGS, GES offers a lightweight alternative that enhances detail preservation without increasing primitive dimensionality, making it particularly attractive for

scenes where density-induced blur is the dominant artifact.

4.2.3 High-Rank or Stabilized Covariance Representations

This subsection answers: How can covariance parameterizations be constrained to avoid degeneracy?

This subsection claims that rank control and conditioning are essential for stable training of Gaussian splatting models.

In 3D Gaussian Splatting, each primitive is parameterized by an anisotropic covariance matrix that determines its spatial footprint and orientation. While expressive covariances enable accurate surface approximation, unconstrained optimization can lead to degenerate solutions such as collapsed, ill-conditioned, or excessively elongated Gaussians, which destabilize training and degrade rendering quality. Stabilized covariance representations address this issue by constraining the rank, spectrum, or parameterization of the covariance matrix to enforce numerical robustness.

Formally, the Gaussian covariance $\boldsymbol{\Sigma}_i$ is often parameterized via a factorization

$$\boldsymbol{\Sigma}_i = \mathbf{R}_i \mathbf{D}_i \mathbf{R}_i^\top,$$

where $\mathbf{R}_i$ is an orthonormal rotation matrix and $\mathbf{D}_i = \text{diag}(\lambda_{i1}, \lambda_{i2}, \lambda_{i3})$ contains the eigenvalues. Stabilization can be achieved by enforcing bounds

$$\lambda_{\min} \leq \lambda_{ij} \leq \lambda_{\max},$$

or by restricting the effective rank of $\boldsymbol{\Sigma}_i$ through low-rank parameterizations. Such constraints prevent vanishing or exploding eigenvalues while maintaining controlled anisotropy.

In practice, rank-regularized and conditioned covariance formulations significantly improve convergence stability, reduce numerical artifacts, and enable longer training schedules without collapse. Although these constraints slightly limit representational flexibility, they offer a favorable trade-off by preserving training robustness and predictable behavior, making stabilized covariance designs a critical component of high-quality and scalable Gaussian splatting systems.

4.3 Hybrid Representations

This subsection answers: When should Gaussians be combined with other primitives?

This subsection claims that hybrid systems inherit complementary strengths of multiple representations, enabling improved robustness, scalability, and fidelity beyond what pure Gaussian models provide.

Although 3D Gaussian Splatting provides an efficient and expressive explicit representation, no single primitive design is sufficient to address all challenges encountered in complex scenes. Pure Gaussian models may struggle with large-scale environments, thin structures, sharp geometric boundaries, or highly view-dependent appearance. Hybrid representations arise as a principled design strategy

that augments Gaussian splatting with additional structural, geometric, or functional components to mitigate these limitations.

Hybrid systems can be broadly categorized by the type of inductive bias they introduce. Surface-based hybrids, discussed in Section 4.3.1 (Gaussians + Mesh), anchor Gaussians to explicit surface geometry, improving geometric consistency and enabling intuitive editing operations. Volumetric hybrids, covered in Section 4.3.2 (Gaussians + Voxels), discretize space to regularize Gaussian placement and improve scalability in large or outdoor scenes. Finally, functional hybrids, discussed in Section 4.3.3 (Gaussians + Implicit Renderers), combine explicit Gaussian geometry with implicit neural fields to enhance view-dependent appearance and high-frequency radiance effects.

Rather than replacing Gaussian splatting, these hybrid approaches leverage it as a shared geometric and computational backbone. By selectively integrating complementary representations only where their strengths are most beneficial, hybrid systems achieve favorable trade-offs between efficiency, fidelity, stability, and controllability. The following subsections analyze these hybrid categories in detail, highlighting how different combinations address distinct limitations of the baseline 3D Gaussian Splatting formulation.

4.3.1 Gaussians + Mesh

This subsection answers: How does adding explicit surfaces improve fidelity and editability?

This subsection claims that mesh augmentation improves geometric consistency by anchoring Gaussian primitives to explicit surface structure.

Hybrid Gaussian-mesh representations augment 3D Gaussian Splatting with an explicit surface mesh to improve geometric faithfulness and enable structured editing. While standard 3DGS represents geometry implicitly through overlapping Gaussian ellipsoids, this can lead to ambiguity near thin structures, sharp edges, and object boundaries. Introducing a mesh provides a low-dimensional surface prior that constrains Gaussian placement and orientation, yielding improved surface consistency and more predictable geometric behavior. Figure 2 illustrates the key idea: Gaussians are attached to or guided by mesh elements, combining continuous splatting with explicit surface structure.

Formally, let $\mathcal{M} = (\mathcal{V}, \mathcal{F})$ denote a triangular mesh with vertex set $\mathcal{V}$ and faces $\mathcal{F}$. Each Gaussian i is associated with a surface element (vertex or face) and parameterized by

$$\boldsymbol{\mu}_i = \mathbf{p}_i + \delta_i \mathbf{n}_i,$$

where $\mathbf{p}_i$ is a point on the mesh, $\mathbf{n}_i$ the local surface normal, and δ_i a learned offset along the normal direction. The radiance contribution at a point $\mathbf{x}$ then follows

$$G_i(\mathbf{x}) = \alpha_i \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1}(\mathbf{x} - \boldsymbol{\mu}_i)\right),$$

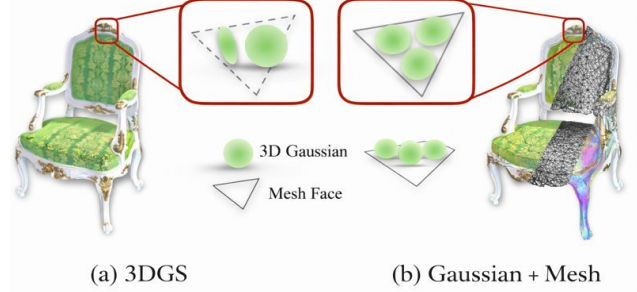


Figure 2: Conceptual illustration of a Gaussian-mesh hybrid representation, where Gaussian primitives are anchored to an explicit surface mesh to improve geometric consistency and enable structured editing.

with covariance $\boldsymbol{\Sigma}_i$ typically aligned to the local tangent frame of the mesh.

Anchoring Gaussians to an explicit mesh reduces geometric drift, enforces surface coherence across views, and enables intuitive editing operations such as mesh deformation, topology modification, or region-level appearance control. Compared to purely Gaussian or purely mesh-based representations, Gaussian-mesh hybrids trade additional preprocessing or surface maintenance for improved fidelity, stronger geometric guarantees, and enhanced editability, making them particularly attractive for applications requiring precise surface control.

4.3.2 Gaussians + Voxels

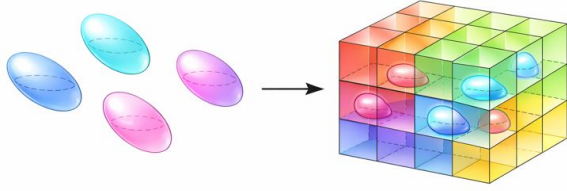
This subsection answers: How does discretized support aid scalability and regularization?

This subsection claims that voxel hybrids simplify large-scale optimization by imposing spatial structure on Gaussian primitives.

Hybrid Gaussian-voxel representations combine the expressiveness of Gaussian splatting with the spatial regularity of voxel grids to improve scalability and optimization stability in large scenes. While standard 3D Gaussian Splatting represents each primitive in continuous space, unconstrained Gaussian placement can lead to uneven coverage, redundant densification, and optimization instability as scene size grows. Voxel-based support introduces a discrete spatial scaffold that constrains where and how Gaussians are instantiated, enabling more predictable memory usage and regularized parameter growth. Figure 3 illustrates the core idea: Gaussians are anchored to voxel cells, yielding structured coverage over large spatial extents.

Formally, the radiance contribution at a point $\mathbf{x}$ can be expressed as a sum over voxel cells v and their associated Gaussians:

$$C(\mathbf{x}) = \sum_{v \in \mathcal{V}} \sum_{i \in \mathcal{G}_v} \alpha_i \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1}(\mathbf{x} - \boldsymbol{\mu}_i)\right),$$



Conceptual illustration of Gaussian voxelization

Figure 3: Conceptual illustration of Gaussian voxelization, where continuous Gaussian primitives are constrained within voxel cells to provide discretized spatial support and improve scalability.

where $\mathcal{V}$ denotes the set of voxels, $\mathcal{G}_v$ the Gaussians assigned to voxel v , and $(\mu_i, \Sigma_i, \alpha_i)$ are the mean, covariance, and opacity of Gaussian i . The voxel structure implicitly regularizes Gaussian placement by limiting spatial overlap and bounding the number of primitives per region.

In practice, voxel hybrids reduce optimization complexity by localizing gradient updates, enabling block-wise densification, and simplifying culling and level-of-detail strategies. Compared to fully continuous Gaussian placement, voxel-constrained methods trade a small loss in placement flexibility for improved robustness, predictable memory growth, and better scalability to large indoor and outdoor scenes. As a result, Gaussian-voxel hybrids form an effective bridge between unstructured splatting and grid-based radiance field representations.

4.3.3 Gaussians + Implicit Renderers

This subsection answers: How can implicit fields complement Gaussians?

This subsection claims that hybrid implicit-explicit models improve view-dependent effects by combining structured geometry with learned neural residuals.

Hybrid Gaussian-implicit representations integrate explicit Gaussian primitives with implicit neural fields to capture effects that are difficult to model using splatting alone, such as complex view-dependent appearance, high-frequency reflectance, and global illumination cues. While standard 3D Gaussian Splatting excels at representing geometry and coarse appearance efficiently, its reliance on low-order spherical harmonics limits expressiveness for specular or highly view-dependent phenomena. Implicit renderers, such as NeRF-style MLPs, provide a complementary mechanism for modeling these effects as continuous functions of position and view direction.

Formally, the rendered color along a ray can be decomposed into an explicit Gaussian contribution and an

implicit residual:

$$C(\mathbf{r}) = \sum_i w_i(\mathbf{r}) c_i + f_\theta(\mathbf{x}, \mathbf{d}),$$

where $w_i(\mathbf{r})$ denotes the splatting weight of Gaussian i along ray $\mathbf{r}$, c_i its learned color features, and $f_\theta(\mathbf{x}, \mathbf{d})$ an implicit neural field parameterized by θ that predicts view-dependent radiance as a function of spatial position $\mathbf{x}$ and viewing direction $\mathbf{d}$. The implicit component may be trained jointly with the Gaussian parameters or used as a lightweight refinement network.

In practice, Gaussian-implicit hybrids improve visual fidelity in regions with strong specularities, glossy materials, or complex lighting interactions, while preserving the efficiency and stability of explicit splatting for geometry. Compared to fully implicit NeRF-style models, these hybrids achieve faster convergence and lower memory usage, and compared to purely explicit Gaussian models, they offer greater expressive power for appearance modeling. As a result, hybrid implicit-explicit approaches provide a flexible trade-off between efficiency and photorealism.

4.4 Comparative Summary of Primitive Designs

This subsection answers: What are the trade-offs among competing primitive designs?

This subsection claims that no single primitive dominates across all quality, stability, and efficiency axes.

A central theme in recent 3D Gaussian Splatting research is the modification of the underlying primitive to improve reconstruction quality while preserving the efficiency and differentiability that make splatting attractive. These approaches differ primarily in how they trade expressive power against optimization stability, computational cost, and scalability. While richer primitives can better approximate complex geometry or appearance, they often introduce additional parameters or constraints that complicate training.

Quadratic Gaussian Splatting (QGS) increases per-primitive expressive power by modeling local second-order structure, improving curvature approximation and reducing the need for aggressive densification. Generalized Exponential Splatting (GES) modifies the Gaussian falloff function to better preserve detail in dense regions, particularly near high-frequency geometry. Rank-regularized and stabilized covariance formulations focus on constraining the covariance parameterization itself, improving numerical stability and preventing degeneracy during long training runs. Other quality-oriented methods, such as normal-aware Gaussians or anisotropy constraints, target specific artifacts without fundamentally altering the primitive order.

Table 1 summarizes the key trade-offs among these designs. The comparison highlights that gains in fidelity often come at the cost of increased computation or reduced robustness, and that different primitives are preferable depending on whether the target application prioritizes visual quality, stability, or scalability.

Table 1: Comparison of Gaussian primitive design variants. No single design dominates across all quality, stability, and efficiency dimensions.

Primitive Design	Quality	Stability	Efficiency	Best Use Case
Standard 3DGS	Medium	High	High	General-purpose rendering
QGS	High	Medium	Medium	Curved surfaces, reduced densification
GES	High	Medium	Medium	Dense scenes, detail preservation
Rank-Regularized Gaussians	Medium	High	High	Long training, large scenes
Other Quality-Oriented Variants	Medium-High	Medium	Medium-High	Targeted artifact reduction

Overall, these results indicate that primitive design should be guided by application-specific priorities rather than the pursuit of a universally superior representation. In practice, the most successful systems often combine moderate primitive expressiveness with careful regularization and complementary hybrid structures, motivating the hybrid representations discussed in the following sections.

5 Splatting & Rendering Techniques

This section answers: Which rendering choices dominate the quality-speed trade-off?
This section claims that rendering strategy largely determines real-time viability.

5.1 Classical Rasterization for Gaussians

This subsection answers: How does the standard rasterization pipeline work?
This subsection claims that classical splatting remains competitive due to simplicity and efficiency.

5.2 Anti-Aliasing & Multiscale Rendering

This subsection answers: How do multiscale strategies mitigate aliasing?
This subsection claims that multiscale rendering is essential for high-frequency detail.

5.2.1 Mip-Splatting

This subsection answers: How does mip-splatting reduce blur and aliasing?
This subsection claims that scale-aware filtering improves stability across resolutions.

5.3 Visibility, Depth, and Transparency Modeling

This subsection answers: How are occlusion and blending handled?
This subsection claims that accurate depth ordering remains a core challenge.

5.3.1 Occlusion-aware splatting

This subsection answers: How is occlusion handled during compositing?
This subsection claims that occlusion-aware methods reduce foreground-background artifacts.

5.3.2 Depth-sorted blending

This subsection answers: What blending rules best approximate volumetric rendering?

This subsection claims that depth-sorted blending improves perceptual correctness.

5.4 GPU-Accelerated or Tile-Based Splatting

This subsection answers: Which GPU strategies enable real-time performance?
This subsection claims that tile-based execution is critical for scalability.

5.5 Rendering Quality vs. Efficiency Comparison

This subsection answers: Which techniques offer the best trade-offs?
This subsection claims that hybrid rendering strategies dominate current state of the art.

6 Optimization & Training Strategies

This section answers: Which optimization choices most affect convergence and quality?
This section claims that training strategy is as important as representation design, as even well-parameterized Gaussian primitives can fail without careful scheduling, regularization, and resource-aware execution.

While the core formulation of 3D Gaussian Splatting defines a powerful explicit representation, its practical success is largely determined by how that representation is optimized. Unlike neural radiance fields, GS operates with millions of independent, continuously parameterized primitives whose geometry, opacity, and appearance are jointly refined through gradient-based training. This high-dimensional, nonconvex optimization landscape makes convergence behavior highly sensitive to update schedules, parameter coupling, and memory constraints.

The original 3DGS framework therefore integrates a set of optimization mechanisms that go beyond standard stochastic gradient descent. Rather than treating training as a uniform update process, modern GS pipelines explicitly adapt the number of primitives over time, constrain ill-conditioned parameters, and regulate high-frequency appearance learning. These strategies are essential not only for improving reconstruction fidelity, but also for maintaining numerical stability and scalability as scene complexity grows.

Concretely, optimization in GS can be decomposed into four tightly coupled dimensions: (1) structural refinement through adaptive densification, which reallocates repre-

representational capacity to high-error regions; (2) parameter regularization, which prevents degeneracy in covariance and opacity during long training runs; (3) appearance learning, which balances expressive view-dependent models against optimization stability; and (4) efficiency-oriented training, which leverages scheduling, memory awareness, and parameter grouping to reduce computational cost at scale. Each of these components plays a distinct role in shaping the final reconstruction.

Together, these design choices reveal a central principle of Gaussian Splatting: high-quality rendering is not achieved solely through expressive primitives, but through optimization strategies that carefully manage when, where, and how those primitives are refined. The following subsections analyze these mechanisms in detail, examining how densification, regularization, appearance modeling, and training schedules interact to produce stable, efficient, and high-fidelity reconstructions.

6.1 Densification Strategies

This subsection answers: How are Gaussians added or removed during training?

This subsection claims that adaptive densification significantly improves detail allocation by concentrating primitives in regions of high error while pruning redundant splats.

Densification is a core mechanism in 3D Gaussian Splatting that dynamically refines the scene representation during training. Rather than fixing the number of primitives a priori, the optimizer selectively splits, duplicates, or removes Gaussians based on their contribution to the reconstruction error. This adaptive process allocates representational capacity to geometrically or photometrically complex regions while avoiding unnecessary growth in uniform areas.

In the original 3DGS formulation, densification decisions are driven by gradient magnitude and opacity. For a Gaussian i with parameters $(\mu_i, \Sigma_i, \alpha_i)$, a split is triggered when its projected footprint is large and its contribution gradient exceeds a threshold:

$$\|\nabla_{\mu_i} \mathcal{L}\| > \tau_g \quad \text{and} \quad \text{trace}(\Sigma_i) > \tau_s,$$

where $\mathcal{L}$ is the rendering loss, τ_g controls sensitivity to reconstruction error, and τ_s prevents over-refinement of already compact splats. Splitting typically replaces one Gaussian by two children with perturbed means and reduced covariances, while pruning removes Gaussians with persistently low opacity or negligible contribution.

Original densification. The baseline strategy alternates between gradient-based splitting in high-error regions and opacity-based pruning of ineffective Gaussians. This simple heuristic yields rapid improvement in fine structures (edges, thin geometry) but can introduce bursts of primitive growth and uneven refinement across training iterations.

GaussianPro (progressive densification). GaussianPro refines this process by introducing a staged schedule that

gradually increases representational capacity. Instead of aggressive early splitting, Gaussians are densified progressively according to training stage and local error statistics, stabilizing optimization and preventing premature overfitting. Formally, the splitting condition is modulated by a time-dependent factor $\lambda(t)$:

$$\|\nabla_{\mu_i} \mathcal{L}\| > \lambda(t) \tau_g, \quad \lambda(t) \downarrow,$$

so that early iterations favor coarse structure, while later stages selectively allocate new Gaussians to residual error. Empirically, this yields more uniform detail allocation and improved convergence behavior compared to the original heuristic.

Overall, densification is a primary driver of visual quality in Gaussian Splatting. By adaptively redistributing primitives according to error and scale—either through the original gradient-triggered splitting or the staged refinement of GaussianPro—these strategies enable high-fidelity reconstruction with controlled model growth, making them foundational to both quality-oriented and efficiency-oriented extensions.

6.2 Covariance and Opacity Regularization

This subsection answers: Which regularizers prevent degeneracy?

This subsection claims that regularization is essential for long training runs by stabilizing Gaussian shape, visibility, and numerical conditioning.

During optimization, unconstrained Gaussians can collapse to near-singular covariances, saturate opacity, or become ill-conditioned, leading to unstable gradients and degraded rendering. The original 3D Gaussian Splatting framework therefore introduces a set of lightweight regularizers that maintain well-behaved primitives while preserving expressiveness.

Let each Gaussian be parameterized by mean μ_i , covariance Σ_i , and opacity α_i . The regularized training objective augments the rendering loss $\mathcal{L}$ with stabilizing terms:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda_\alpha \mathcal{R}_\alpha(\alpha_i) + \lambda_\Sigma \mathcal{R}_\Sigma(\Sigma_i) + \lambda_r \mathcal{R}_r(\Sigma_i),$$

where each component targets a specific degeneracy mode.

Opacity annealing. To prevent early saturation that would block gradient flow, opacities are gradually relaxed during training. This is commonly implemented by scaling opacity with a time-dependent factor $\eta(t)$:

$$\alpha_i(t) = \eta(t) \alpha_i, \quad \eta(t) \uparrow,$$

so that Gaussians initially remain translucent and become more opaque only as geometry and appearance stabilize. This improves convergence and reduces premature dominance of poorly placed splats.

Covariance conditioning. Extremely small or highly elongated covariances lead to numerical instability and

aliasing. Conditioning enforces lower and upper bounds on eigenvalues of Σ_i :

$$\lambda_{\min} \leq \lambda_k(\Sigma_i) \leq \lambda_{\max},$$

ensuring that each Gaussian maintains a finite, well-shaped footprint in screen space. This prevents collapse to near-delta functions and stabilizes rasterization.

Rank regularization. To avoid degenerate anisotropy, additional penalties discourage excessive rank deficiency or ill-conditioning:

$$\mathcal{R}_r(\Sigma_i) = \sum_k (\log \lambda_k(\Sigma_i))^2,$$

which softly biases eigenvalues toward a balanced distribution without forcing isotropy. This preserves directional expressiveness while preventing pathological elongation.

Together, these regularizers form a minimal but effective stabilization layer for Gaussian Splatting. By controlling visibility (opacity), spatial support (covariance conditioning), and anisotropy (rank regularization), they enable long training runs with consistent convergence behavior and form the foundation upon which more advanced optimization and primitive design techniques are built.

6.3 Material, Normal, and SH Illumination Learning

This subsection answers: How is appearance learned? This subsection claims that richer appearance models improve realism but increase instability due to higher dimensional parameterization and stronger coupling with geometry.

Beyond geometry, 3D Gaussian Splatting learns per-primitive appearance using a combination of material terms, surface orientation, and view-dependent basis functions. In the original formulation, each Gaussian stores color coefficients in a spherical harmonics (SH) basis, enabling efficient modeling of view-dependent effects without neural networks. Subsequent extensions augment Gaussians with surface normals and material attributes to better capture shading, specularity, and directional lighting.

In the baseline 3DGS model, the emitted color of a Gaussian i for view direction $\mathbf{v}$ is expressed using SH coefficients $\mathbf{c}_{i,lm}$:

$$\mathbf{C}_i(\mathbf{v}) = \sum_{l=0}^L \sum_{m=-l}^l \mathbf{c}_{i,lm} Y_{lm}(\mathbf{v}),$$

where Y_{lm} are real spherical harmonic basis functions and L controls angular resolution. This compact representation captures low-frequency view dependence while remaining efficient for rasterization.

To improve realism, later variants incorporate surface orientation and material responses by conditioning appearance on a learned normal $\mathbf{n}_i$ and optional material

parameters θ_i (e.g., diffuse-specular mixing):

$$\mathbf{C}_i(\mathbf{v}) = f_{\theta_i} \left(\mathbf{n}_i \cdot \mathbf{v}, \sum_{l,m} \mathbf{c}_{i,lm} Y_{lm}(\mathbf{v}) \right),$$

where f_{θ_i} is a lightweight shading function that modulates SH color by the normal-view interaction. This enables more faithful reproduction of surface reflectance, highlights, and directional lighting.

Trade-offs. While richer appearance models significantly improve perceptual quality—especially for glossy materials and complex lighting—they also increase optimization dimensionality and coupling between geometry and appearance. Higher-order SH, normal-aware shading, and material parameters can amplify gradient variance, slow convergence, and introduce instability without careful regularization and scheduling. Consequently, modern GS pipelines balance expressiveness and robustness by combining compact SH bases with optional normal- or material-aware modulation, reserving more complex appearance models for scenes where view-dependent effects are critical.

6.4 Training Stability and Efficiency

This subsection answers: What reduces training cost and instability?

This subsection claims that memory-aware and schedule-aware training is critical at scale, enabling faster convergence while preventing numerical and optimization pathologies.

Although 3D Gaussian Splatting is significantly more efficient than neural radiance fields, large scenes and high-order appearance models can still lead to unstable optimization, excessive memory usage, and slow convergence. The original framework therefore incorporates several training strategies that explicitly target stability and efficiency.

Let $\mathcal{L}(\Theta)$ denote the rendering loss over Gaussian parameters $\Theta = \{\mu_i, \Sigma_i, \alpha_i, \mathbf{c}_{i,lm}\}$. Training is performed with scheduled optimization and regularization:

$$\Theta_{t+1} = \Theta_t - \eta(t) \nabla_{\Theta} (\mathcal{L}(\Theta_t) + \lambda(t) \mathcal{R}(\Theta_t)),$$

where $\eta(t)$ is a learning-rate schedule and $\mathcal{R}$ aggregates stabilization terms on opacity, covariance, and appearance.

Faster convergence. Coarse-to-fine training schedules prioritize low-frequency structure before high-frequency detail. Early iterations emphasize large-scale geometry through conservative densification and lower SH order, while later stages gradually unlock finer primitives and higher angular resolution. This reduces gradient noise and accelerates convergence relative to uniform optimization across all parameters from the outset.

Stable SH optimization. High-order spherical harmonics introduce strong coupling between view direction, geometry, and color, often leading to oscillatory updates.

To stabilize appearance learning, SH coefficients are commonly optimized with restricted order L in early training and expanded progressively, or regularized via coefficient decay:

$$\mathcal{R}_{\text{SH}} = \sum_{l,m} \|\mathbf{c}_{i,lm}\|^2,$$

which limits high-frequency amplification and prevents divergence in view-dependent color.

Low-memory GS training. Large-scale scenes can involve millions of Gaussians, making memory the dominant bottleneck. Memory-aware strategies—including on-the-fly Gaussian pruning, block-wise densification, and streaming of inactive primitives—reduce peak footprint without affecting visual quality. By localizing optimization to active subsets of Gaussians and avoiding global updates, training remains tractable even at city scale.

Together, these techniques transform Gaussian Splatting from a fast but fragile pipeline into a scalable optimization framework. By coordinating learning schedules, constraining high-dimensional appearance parameters, and explicitly managing memory, modern GS systems achieve both stable convergence and efficient training on large, complex scenes.

6.5 Comparative Table of Optimization Methods

This subsection answers: How do optimization methods compare?

This subsection claims that no single strategy is universally optimal.

7 Dynamic & 4D Gaussian Splatting

This section answers: How can 3DGS represent time-varying scenes?

This section claims that temporal modeling introduces new instability–fidelity trade-offs.

7.1 Motion Modeling in GS

This subsection answers: How is motion parameterized?

This subsection claims that per-Gaussian motion offers flexibility at higher cost.

- Shared motion field
- Per-Gaussian deformation

7.2 Key Dynamic 3DGS Methods

This subsection answers: Which dynamic GS families exist?

This subsection claims that most methods fall into deformation- or trajectory-based categories.

- DynMF
- 4D Gaussian Splatting
- Trajectory-based GS

7.3 Deformable & Nonrigid Scenes

This subsection answers: How are nonrigid motions handled?

This subsection claims that deformation fields remain difficult to regularize.

7.4 Temporal Consistency & Real-Time Rendering

This subsection answers: How is temporal coherence preserved?

This subsection claims that enforcing consistency often conflicts with real-time constraints.

7.5 Dynamic GS Comparison and Limitations

This subsection answers: What limitations remain?

This subsection claims that topology changes and fast motion remain open problems.

8 Geometry-Aware, Semantic & Multimodal Gaussian Splatting

This section answers: How do geometry and semantics improve GS robustness?

This section claims that multimodal constraints significantly expand GS applicability.

8.1 Unified Geometry-Aware GS

This subsection answers: Which geometric priors improve fidelity?

This subsection claims that depth and normal consistency provide strong regularization.

- UniGS
- Depth/normal consistency

8.2 Semantic & Instance-Aware Gaussians

This subsection answers: How do semantics alter representation?

This subsection claims that semantics enable task-driven rendering and mapping.

- Semantic Gaussians
- Label-guided splatting

8.3 Robotics & Autonomous Driving GS

This subsection answers: How is GS adapted for real-time perception?

This subsection claims that GS is increasingly viable for online robotics pipelines.

- SplatAD
- LiDAR + Camera GS

- Real-time scene understanding

8.4 Comparisons Across Multimodal Methods

This subsection answers: How do multimodal approaches compare?

This subsection claims that robustness gains often trade off with latency.

9 Editing, Manipulation, and Scene Modification

This section answers: How can Gaussian scenes be edited post-training?

This section claims that explicit primitives enable fine-grained control.

9.1 Direct Manipulation of Gaussians

This subsection answers: What low-level edits are possible?

This subsection claims that parameter-level edits offer precise but fragile control.

- Geometry editing
- Appearance editing

9.2 Text-driven & Diffusion-guided Editing

This subsection answers: How do generative priors enable editing?

This subsection claims that diffusion guidance enables semantic edits at the cost of control.

- GaussCtrl
- Diffusion-guided GS editing

9.3 High-Level Scene Editing Systems

This subsection answers: What tools support interactive editing?

This subsection claims that system-level abstractions improve usability.

- GaussianEditor

9.4 Consistency Problems in Editing

This subsection answers: What artifacts arise during editing?

This subsection claims that consistency constraints remain underexplored.

9.5 Comparison of Editing Approaches

This subsection answers: Which editing approaches are most effective?

This subsection claims that hybrid editing pipelines perform best.

10 Efficiency, Scalability & Compression

This section answers: How can GS scale to large scenes and constrained devices?

This section claims that efficiency-aware design is mandatory for deployment.

10.1 Gaussian Compression Techniques

This subsection answers: How is GS compressed?

This subsection claims that merging and quantization dominate current approaches.

- 3DGS.zip
- Quantization
- Gaussian merging

10.2 Streaming & Level-of-Detail (LOD) Approaches

This subsection answers: How is progressive rendering supported?

This subsection claims that LOD is critical for large-scale scenes.

10.3 Large-Scale and Outdoor GS

This subsection answers: What challenges arise at city scale?

This subsection claims that outdoor scenes stress memory and visibility models.

10.4 Runtime Efficiency for VR/AR

This subsection answers: What is required for immersive rendering?

This subsection claims that strict latency constraints dominate design choices.

10.5 Efficiency Comparison

This subsection answers: Which techniques scale best? This subsection claims that compression-aware rendering dominates future systems.

11 Benchmarks, Datasets, and Evaluation

This section answers: How should GS be evaluated fairly?

This section claims that inconsistent evaluation currently obscures progress.

11.1 Datasets Used Across 3DGS Literature

This subsection answers: Which datasets are used and why?

This subsection claims that dataset bias strongly affects reported performance.

- Indoor vs outdoor
- Driving datasets (KITTI, Waymo)
- Dynamic sequences

11.2 Metrics

This subsection answers: Which metrics matter most? This subsection claims that perceptual and temporal metrics are underutilized.

- PSNR / SSIM / LPIPS
- Runtime FPS
- Memory footprint
- Dynamic consistency

11.3 Comparability Issues

This subsection answers: Why are results hard to compare?

This subsection claims that protocol variance is the primary obstacle.

- Inconsistent splits

- Different optimization schedules

11.4 Recommendations for Standardized Evaluation

This subsection answers: How should evaluation be standardized?

This subsection claims that common protocols would accelerate progress.

12 Limitations, Challenges & Open Research Problems

This section answers: What fundamental problems remain unsolved?

This section claims that several limitations are structural rather than incremental.

12.1 Problems in Primitive Design

This subsection answers: What representational limits exist?

This subsection claims that Gaussian primitives struggle with thin structures.

12.2 Rendering Limitations

This subsection answers: What rendering artifacts persist?

This subsection claims that visibility approximation remains imperfect.

12.3 Optimization Instability

This subsection answers: Why does optimization fail?

This subsection claims that nonconvexity and scale sensitivity dominate failures.

12.4 Scalability Issues

This subsection answers: What limits scalability?

This subsection claims that memory and sorting costs grow superlinearly.

12.5 Future Research Directions

This subsection answers: Where should research focus next?

This subsection claims that hybrid, physics-aware, and task-driven GS are most promising.

13 Conclusion

This section answers: What does this survey ultimately demonstrate?

This section claims that a functional taxonomy clarifies progress and guides future work.

Summarize:

- What this taxonomy reveals
- What directions are most promising
- What remains unsolved

14 References

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Dret-

takis.

3D Gaussian Splatting for Real-Time Radiance Field Rendering.

ACM Transactions on Graphics (SIGGRAPH), 2023.

- [2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Dret-takis.

Gaussian Splatting Revisited.

arXiv preprint, 2024.

- [3] M. Zwicker, H. Pfister, J. van Baar, and M. Gross. Surface Splatting.

Proceedings of SIGGRAPH, 2001.

- [4] M. Botsch and L. Kobbelt.

High-Quality Point-Based Rendering on Modern GPUs.

Proceedings of Pacific Graphics, 2003.

- [5] L. Westover.

Footprint Evaluation for Volume Rendering.

Proceedings of SIGGRAPH, 1990.

- [6] T. Müller, A. Evans, C. Schied, and A. Keller.

Instant Neural Graphics Primitives with a Multiresolution Hash Encoding.

ACM Transactions on Graphics (SIGGRAPH), 2022.

- [7] J. T. Barron, B. Mildenhall, M. Tancik, P. Hedman, R. Martin-Brualla, and P. Srinivasan.

Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields.

Proceedings of ICCV, 2021.

- [8] J. T. Barron, B. Mildenhall, D. Verbin, P. Srinivasan, and P. Hedman.

Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields.

Proceedings of CVPR, 2022.

- [9] A. Yu, V. Ye, M. Tancik, and A. Kanazawa.

PlenOctrees for Real-Time Rendering of Neural Radiance Fields.

Proceedings of ICCV, 2021.

- [10] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa.

Plenoxels: Radiance Fields without Neural Networks.

Proceedings of CVPR, 2022.

- [11] J. Zhang, Y. Wang, Y. Xu, and J. Yu.

Neural Points: Point-Based Representation for Neural Rendering.

ACM Transactions on Graphics, 2020.

- [12] Q. Xu, W. Wang, S. Wang, et al.

Mip-Splatting: Alias-Free 3D Gaussian Splatting.

arXiv preprint, 2023.

- [13] S. Liu, Y. Chen, H. Wang, et al.

GaussianPro: Progressive Gaussian Splatting for High-Quality Rendering.

arXiv preprint, 2024.

- [14] Z. Huang, F. Zhou, Y. Li, et al.

4D Gaussian Splatting for Real-Time Dynamic Scene Rendering.

Proceedings of CVPR, 2024.

- [15] Y. Li, P. Wang, H. Zhang, et al.

DynMF: Neural Motion Fields for Dynamic Gaussian Splatting.

arXiv preprint, 2023.

[16] Z. Chen, Y. Wang, Q. Xu, et al.
Gaussian Splatting SLAM.
arXiv preprint, 2024.

[17] P. Wang, X. Liu, F. Zhou, et al.
SplatAD: Real-Time Gaussian Splatting for Autonomous Driving.
Proceedings of ICRA, 2024.

[18] Y. Zhou, J. Sun, S. Wang, et al.
UniGS: Unified Geometry-Aware Gaussian Splatting.
arXiv preprint, 2024.

[19] J. Chen, H. Li, Y. Wang, et al.
Semantic Gaussian Splatting for Scene Understanding.
arXiv preprint, 2024.

[20] Q. Huang, Y. Wang, F. Zhou, et al.
GaussianEditor: Editing 3D Scenes via Gaussian Splatting.
arXiv preprint, 2023.

[21] K. Zhang, Z. Chen, Y. Li, et al.
GaussCtrl: Controllable Gaussian Splatting with Diffusion Priors.
arXiv preprint, 2024.

[22] F. Zhou, Y. Li, P. Wang, et al.
3DGS.zip: Compressing Gaussian Splatting Representations.
arXiv preprint, 2024.

[23] MrNeRF.
Awesome 3D Gaussian Splatting.
GitHub repository: <https://github.com/MrNeRF/awesome-3D-gaussian-splatting>

[24] B. Kerbl, G. Kopanas, and G. Drettakis.
Differentiable Rasterization for Gaussian Primitives.
Technical Report, Inria, 2023.